AWS Cost-Cutting Checklist for Indian SMBs

A practical 30-item checklist used in real engagements. Most Indian SMBs running AWS find 20–30% savings within the first month by working through these. No tool subscriptions required — every check below uses the AWS Console, AWS CLI, or open-source tooling.

> Built and maintained by AICloudStrategist · Founder-led. Enterprise-reviewed. · support@aicloudstrategist.com

---

How to use

Spend 2–3 hours running through this end to end. Make a spreadsheet with three columns: *Check · Status · Estimated monthly savings (INR)*. Don't fix anything yet — first build the picture. Most teams underestimate savings before they look.

If you find more than 5 high-value items in 2 hours, that's a signal you need a structured FinOps engagement. Book a free 30-min call.

Section 1 — Compute (EC2, EKS, Fargate)

Quick wins. Look here first.

1. **Idle EC2 instances** — Cost Explorer → Resource Optimization → Idle Recommendations. Anything with <5% CPU and <5MB/s network for 14+ days is dead weight.
2. **Stopped instances with attached EBS** — they don't bill compute but EBS keeps charging. Snapshot + terminate, don't just stop.
3. **Right-size oversized instances** — Compute Optimizer → look for "over-provisioned" recommendations. Resize before considering RIs.
4. **Old generation instances** — m4 → m6, c4 → c6, etc. Newer generation is cheaper per vCPU and faster.
5. **Spot for non-prod** — dev/staging/CI runners belong on Spot. 70–90% cheaper.
6. **EKS node groups sized for peak QPS** — enable cluster autoscaler + Karpenter. Check `kubectl top nodes` over 7 days.
7. **Fargate vs EC2 for low-frequency jobs** — Fargate wins for ad-hoc; EC2 wins for steady workloads >70% utilized.
8. **Scheduled stop/start for non-prod** — AWS Instance Scheduler, free. Stop dev environments nights + weekends → ~70% savings on those.

Section 2 — Storage (S3, EBS, EFS)

9. **S3 lifecycle policies** — anything older than 30 days → Standard-IA; older than 90 → Glacier Instant; older than 365 → Glacier Deep Archive.
10. **S3 Intelligent-Tiering for unknown access patterns** — set as default for new buckets.
11. **Incomplete multipart uploads** — set lifecycle rule to

delete after 7 days. Free win.

12. **Orphaned EBS volumes** — `aws ec2 describe-volumes --filters Name=status,Values=available`. These cost money for no benefit.
13. **EBS gp2 → gp3** — same performance, ~20% cheaper. One-line modification.
14. **Old EBS snapshots** — snapshots from terminated instances. List → review → delete the truly stale ones.
15. **EFS Infrequent Access** — turn on lifecycle policy to move to IA after 30 days.

Section 3 — Network (Data transfer, NAT, VPC endpoints)

16. **NAT Gateway is expensive** (~₹3,500/month + per-GB). For high-egress workloads, replace with VPC Endpoints (Gateway endpoints for S3/DynamoDB are FREE).
17. **Cross-AZ traffic** — check Cost Explorer → Group by Usage Type → look for `DataTransfer-Regional-Bytes`. Co-locate chatty services.
18. **CloudFront in front of S3 + ALB** — caches at edge, dramatically cuts origin traffic for India audiences.
19. **VPC Endpoints** for S3, DynamoDB, ECR, Secrets Manager, SSM — free (Gateway type) or much cheaper than NAT (Interface type).
20. **Inspect your egress to internet** — the most expensive byte type. Caching + compression often pays back in weeks.

Section 4 — Databases (RDS, Aurora, ElastiCache)

21. **Right-size RDS** — RDS Performance Insights shows whether you're CPU-bound, IO-bound, or idle. Most teams over-provision by 1–2 sizes.
22. **Aurora Serverless v2 for spiky workloads** — scales to zero in dev/test.
23. **Single-AZ for non-prod RDS** — Multi-AZ doubles your cost; only prod needs it.
24. **Reserved Instances after 30 days of stable load** — 1-year RI ~30% off, 3-year ~55% off. Don't commit before measuring baseline.
25. **gp2 → gp3 for RDS storage** — same as EBS; 20% cheaper.

Section 5 — Reserved Instances + Savings Plans

26. **Compute Savings Plans cover EC2 + Fargate + Lambda** — the most flexible RI-style commitment. Start with 1-year, no-upfront on your stable baseline.
27. **AWS RI / SP recommendations** — Cost Explorer → Reservations → Recommendations. Don't blindly accept; check that the baseline is real.
28. **Don't over-commit** — uncovered Spot/on-demand capacity is fine. Aim for 60–70% RI/SP coverage on stable workloads, leave headroom.

Section 6 — Tagging + Governance

29. **Mandate cost-allocation tags** — minimum: `env`, `owner`, `service`, `cost_center`. Without these, you can't allocate or chase down waste.
30. **Enable AWS Cost Anomaly Detection** — free, alerts when

spend deviates from baseline. Catches the “we left a t3.2xlarge running for a week” mistakes.

What “20–30% savings” actually looks like

Real numbers from a recent engagement (D2C e-commerce, ~₹35L/month AWS):

Action	Monthly savings
Right-sized 60% of EC2 fleet (oversized post-holiday scale-up)	₹4.2L
S3 lifecycle policy on cold archives → IA + Glacier	₹1.8L
Removed orphaned EBS volumes from old terminations	₹0.6L
Replaced NAT Gateway with VPC endpoints for S3 + ECR	₹0.9L
1-year Compute Savings Plan on baseline	₹1.9L
Total verified savings	**₹9.4L/month**

That’s 27% reduction. Implementation took 3 weeks of part-time work.

The trap: visibility without action

Most teams can find waste. Few have the discipline to act on it monthly. The ROI of a FinOps practice isn’t the first sweep — it’s the *standing capability* that catches drift before it accumulates.

If you want help making this a habit instead of a one-off:

Book a free 30-min Cloud Cost Health Check → 30 minutes. Free. You leave with the top 3 things to fix this week, even if we never work together.

— Anushka B, Founder · AICloudStrategist
support@aicloudstrategist.com